Provide the solutions for all the MCD questions provide both questions with answers

Hey bro! 👊 I've got the **144 MongoDB / NoSQL MCQs** from your uploaded Excel file. I'll give you **every question with all four options + the correct answer**, so you can use it directly for exam preparation.

Unit V – NoSQL & MongoDB — 144 MCQs

1–20

1. NoSQL is commonly expanded as:
A) No Structured Query Language
B) Non-relational Standard Query Language
C) Network Oriented SQL
D) Not Only SQL
✅ **Answer: D) Not Only SQL**

2. Which of the following is NOT a type of NoSQL data store?
A) Tabular relational store with fixed schema
B) Key-value store
C) Document store
D) Graph database
✅ **Answer: A) Tabular relational store with fixed schema**

3. Redis is an example of which type of NoSQL store?
A) Document store
B) Graph database
C) Key-value store
D) Relational database
✅ **Answer: C) Key-value store**

4. Apache Cassandra belongs to which category of NoSQL databases?
A) Document store
B) Graph database
C) Key-value store
D) Wide-column (column-family) store
✅ **Answer: D) Wide-column (column-family) store**

5. Neo4j is a popular example of a:
A) Graph database
B) Document database
C) Wide-column store
D) Key-value store
✅ **Answer: A) Graph database**

6. MongoDB is classified as a:
A) Graph database
B) Key-value store
C) Relational database
D) Document-oriented database
✅ **Answer: D) Document-oriented database**

7. Compared with SQL databases, NoSQL databases typically offer:
A) A flexible, dynamic schema
B) A rigid, fixed schema
C) Mandatory multi-table JOINs
D) Support for vertical scaling only
✅ **Answer: A) A flexible, dynamic schema**

8. NoSQL databases are generally designed to scale:
A) Only by upgrading a single server's hardware
B) Only by adding more tables
C) Horizontally by adding commodity servers
D) By eliminating data replication
✅ **Answer: C) Horizontally by adding commodity servers**

9. The CAP theorem was proposed by:
A) E. F. Codd
B) Michael Stonebraker
C) Eric Brewer
D) Jim Gray
✅ **Answer: C) Eric Brewer**

10. In the CAP theorem, 'C' stands for Consistency, which means:
A) Every request receives a non-error response
B) Every read receives the most recent write or an error
C) The system keeps working despite network failures
D) Data is compressed before storage
✅ **Answer: B) Every read receives the most recent write or an error**

11. In the CAP theorem, 'A' stands for:
A) Availability
B) Authentication
C) Atomicity
D) Adaptability
✅ **Answer: A) Availability**

12. In the CAP theorem, 'P' stands for:
A) Performance
B) Partition tolerance
C) Persistence

D) Processing
✅ **Answer: B) Partition tolerance**

13. MongoDB stores data primarily as:
A) Rows
B) XML files
C) BSON documents
D) CSV files
✅ **Answer: C) BSON documents**

14. BSON stands for:
A) Binary Structured Object Notation
B) Binary JSON
C) Basic Serialized Object Notation
D) Binary SQL Object Notation
✅ **Answer: B) Binary JSON**

15. A MongoDB database contains:
A) Tables only
B) Collections
C) Worksheets
D) Tuples
✅ **Answer: B) Collections**

16. A MongoDB collection is roughly equivalent to a:
A) Table in a relational database
B) Column in a relational database
C) Row in a relational database
D) Primary key
✅ **Answer: A) Table in a relational database**

17. A MongoDB document is roughly equivalent to a:
A) Database
B) Table
C) Row/record
D) Column
✅ **Answer: C) Row/record**

18. MongoDB documents are stored in:
A) JSON format only
B) BSON format
C) CSV format
D) XML format
✅ **Answer: B) BSON format**

19. Which field uniquely identifies a MongoDB document by default?
A) id

B) key
C) _id
D) primary_key
✅ **Answer: C) _id**

20. If `_id` is not provided when inserting a MongoDB document:
A) The insertion fails
B) MongoDB automatically generates an ObjectId
C) MongoDB uses the collection name
D) MongoDB generates a NULL value
✅ **Answer: B) MongoDB automatically generates an ObjectId**

21–40

21. Which command is commonly used to show MongoDB databases?
A) `show dbs`
B) `list databases`
C) `show databases`
D) `db.show()`
✅ **Answer: A) `show dbs`**

22. Which command switches to or creates a database context in MongoDB?
A) `use database_name`
B) `switch database_name`
C) `open database_name`
D) `select database_name`
✅ **Answer: A) `use database_name`**

23. Which command displays the current MongoDB database?
A) `db.current()`
B) `show current`
C) `db`
D) `currentdb()`
✅ **Answer: C) `db`**

24. Which command displays collections in the current database?
A) `show collections`
B) `list tables`
C) `show tables`
D) `db.collections()`
✅ **Answer: A) `show collections`**

25. Which method is used to insert a single document?
A) `insertOne()`
B) `insertSingle()`
C) `addOne()`

D) `createOne()`
✅ **Answer: A) `insertOne()`**

26. Which method is used to insert multiple documents?
A) `insertMany()`
B) `insertMultiple()`
C) `addMany()`
D) `createMany()`
✅ **Answer: A) `insertMany()`**

27. Which method retrieves documents from a collection?
A) `get()`
B) `select()`
C) `find()`
D) `read()`
✅ **Answer: C) `find()`**

28. Which MongoDB method retrieves only one matching document?
A) `findOne()`
B) `findSingle()`
C) `getOne()`
D) `selectOne()`
✅ **Answer: A) `findOne()`**

29. Which method is used to update one document?
A) `changeOne()`
B) `updateOne()`
C) `modifyOne()`
D) `editOne()`
✅ **Answer: B) `updateOne()`**

30. Which method updates multiple matching documents?
A) `updateMany()`
B) `modifyMany()`
C) `changeMany()`
D) `editMany()`
✅ **Answer: A) `updateMany()`**

31. Which method deletes one matching document?
A) `removeOne()`
B) `deleteOne()`
C) `dropOne()`
D) `eraseOne()`
✅ **Answer: B) `deleteOne()`**

32. Which method deletes multiple matching documents?
A) `deleteMany()`

B) `removeMany()`
C) `dropMany()`
D) `eraseMany()`
✅ **Answer: A) `deleteMany()`**

33. Which command removes an entire collection?
A) `deleteCollection()`
B) `removeCollection()`
C) `db.collection.drop()`
D) `db.collection.delete()`
✅ **Answer: C) `db.collection.drop()`**

34. Which command removes the current database?
A) `db.dropDatabase()`
B) `db.deleteDatabase()`
C) `drop.db()`
D) `deleteDatabase()`
✅ **Answer: A) `db.dropDatabase()`**

35. Which MongoDB operator is used for equality?
A) `$eq`
B) `$equal`
C) `$equals`
D) `$same`
✅ **Answer: A) `$eq`**

36. Which operator means “not equal to”?
A) `$neq`
B) `$ne`
C) `$notequal`
D) `$notEqual`
✅ **Answer: B) `$ne`**

37. Which operator checks whether a value is greater than another value?
A) `$gt`
B) `$greater`
C) `$more`
D) `$greaterThan`
✅ **Answer: A) `$gt`**

38. Which operator means “greater than or equal to”?
A) `$gte`
B) `$ge`
C) `$greaterEqual`
D) `$gteq`
✅ **Answer: A) `$gte`**

39. Which operator means “less than”?
A) `$lt`
B) `$less`
C) `$lower`
D) `$lessThan`
✅ **Answer: A) `$lt`**

40. Which operator means “less than or equal to”?
A) `$lte`
B) `$le`
C) `$lessEqual`
D) `$lteq`
✅ **Answer: A) `$lte`**

41–60

41. Which operator is used to match any value in an array of specified values?
A) `$in`
B) `$any`
C) `$match`
D) `$contains`
✅ **Answer: A) `$in`**

42. Which operator matches values that are not in a specified array?
A) `$notIn`
B) `$nin`
C) `$not`
D) `$exclude`
✅ **Answer: B) `$nin`**

43. Which logical operator performs an AND operation?
A) `$and`
B) `$all`
C) `$both`
D) `$&&`
✅ **Answer: A) `$and`**

44. Which logical operator performs an OR operation?
A) `$or`
B) `$either`
C) `$any`
D) `$||`
✅ **Answer: A) `$or`**

45. Which operator performs logical NOT?
A) `$not`

B) `$!`
C) `$negate`
D) `$no`
✅ **Answer: A) `$not`**

46. Which operator performs logical NOR?
A) `$nor`
B) `$none`
C) `$notOr`
D) `$norOp`
✅ **Answer: A) `$nor`**

47. Which operator checks whether a field exists?
A) `$exists`
B) `$present`
C) `$has`
D) `$field`
✅ **Answer: A) `$exists`**

48. Which operator is commonly used for regular expression matching?
A) `$regex`
B) `$match`
C) `$pattern`
D) `$search`
✅ **Answer: A) `$regex`**

49. Which operator checks the data type of a field?
A) `$datatype`
B) `$type`
C) `$typeof`
D) `$dataType`
✅ **Answer: B) `$type`**

50. Which operator matches an array containing all specified elements?
A) `$contains`
B) `$all`
C) `$every`
D) `$include`
✅ **Answer: B) `$all`**

51. Which operator matches an array with a specified size?
A) `$length`
B) `$size`
C) `$count`
D) `$arraySize`
✅ **Answer: B) `$size`**

52. Which operator checks whether at least one array element satisfies a condition?

A) `$elemMatch`

B) `$arrayMatch`

C) `$element`

D) `$matchArray`

✅ **Answer: A) `$elemMatch`**

53. Which update operator increments a numeric field?

A) `$increase`

B) `$inc`

C) `$add`

D) `$increment`

✅ **Answer: B) `$inc`**

54. Which operator multiplies a numeric field by a specified value?

A) `$multiply`

B) `$mul`

C) `$times`

D) `$product`

✅ **Answer: B) `$mul`**

55. Which operator removes a field from a document?

A) `$delete`

B) `$remove`

C) `$unset`

D) `$drop`

✅ **Answer: C) `$unset`**

56. Which operator adds a field or updates its value?

A) `$set`

B) `$addField`

C) `$field`

D) `$put`

✅ **Answer: A) `$set`**

57. Which operator adds an element to an array?

A) `$addToSet`

B) `$push`

C) Both A and B depending on requirement

D) `$appendOnly`

✅ **Answer: C) Both A and B depending on requirement**

58. What is the main difference between `$push` and `$addToSet`?

A) `$push` prevents duplicates

B) `$addToSet` prevents duplicate values

C) Both always prevent duplicates

D) Neither can modify arrays
✅ **Answer: B) `$addToSet` prevents duplicate values**

59. Which operator removes the first or last element from an array?
A) `$pop`
B) `$remove`
C) `$shift`
D) `$deleteArray`
✅ **Answer: A) `$pop`**

60. Which operator removes matching array elements?
A) `$pull`
B) `$remove`
C) `$delete`
D) `$filterOut`
✅ **Answer: A) `$pull`**

61–80

61. Which method is used to sort query results?
A) `sort()`
B) `order()`
C) `arrange()`
D) `sequence()`
✅ **Answer: A) `sort()`**

62. In MongoDB, a sort value of 1 means:
A) Descending order
B) Ascending order
C) Random order
D) No sorting
✅ **Answer: B) Ascending order**

63. In MongoDB, a sort value of `-1` means:
A) Ascending order
B) Descending order
C) Random order
D) Reverse insertion order
✅ **Answer: B) Descending order**

64. Which method limits the number of documents returned?
A) `limit()`
B) `restrict()`
C) `max()`
D) `countLimit()`
✅ **Answer: A) `limit()`**

65. Which method skips a specified number of documents?

A) `skip()`

B) `ignore()`

C) `jump()`

D) `offset()`

✅ **Answer: A) `skip()`**

66. Which method counts documents in a collection/query?

A) `countDocuments()`

B) `countRows()`

C) `countRecords()`

D) `totalDocuments()`

✅ **Answer: A) `countDocuments()`**

67. Projection in MongoDB is used to:

A) Modify documents

B) Select specific fields from documents

C) Delete documents

D) Create indexes

✅ **Answer: B) Select specific fields from documents**

68. In MongoDB projection, a value of `1` generally means:

A) Exclude the field

B) Include the field

C) Delete the field

D) Sort the field

✅ **Answer: B) Include the field**

69. In MongoDB projection, a value of `0` generally means:

A) Include the field

B) Exclude the field

C) Sort ascending

D) Make the field null

✅ **Answer: B) Exclude the field**

70. Which field is normally excluded from a projection when needed?

A) `id`

B) `_id`

C) `primary`

D) `key`

✅ **Answer: B) `_id`**

71. An index in MongoDB is primarily used to:

A) Increase query performance

B) Store backups

C) Compress documents

D) Replace collections
✅ **Answer: A) Increase query performance**

72. Which method creates an index?
A) `createIndex()`
B) `makeIndex()`
C) `addIndex()`
D) `newIndex()`
✅ **Answer: A) `createIndex()`**

73. Which command lists indexes on a collection?
A) `showIndexes()`
B) `listIndexes()`
C) `getIndexes()`
D) All of the above can be context-dependent methods, but `getIndexes()` is commonly used
✅ **Answer: D) All of the above can be context-dependent methods, but `getIndexes()` is commonly used**

74. Which type of index is automatically created for the `_id` field?
A) Unique index
B) Text index
C) Hashed index
D) Sparse index only
✅ **Answer: A) Unique index**

75. A unique index ensures that:
A) Every document has the same value
B) Indexed values are not duplicated
C) Documents are stored in alphabetical order
D) Fields cannot be null
✅ **Answer: B) Indexed values are not duplicated**

76. Which MongoDB index is designed for text search?
A) Text index
B) Search index
C) String index
D) Word index
✅ **Answer: A) Text index**

77. Which MongoDB index type is useful for hashed shard keys?
A) Hashed index
B) Text index
C) Sparse index
D) TTL index
✅ **Answer: A) Hashed index**

78. TTL indexes are primarily used to:
A) Automatically expire documents
B) Speed up text search
C) Encrypt documents
D) Create relationships
✅ **Answer: A) Automatically expire documents**

79. What does CRUD stand for?
A) Create, Read, Update, Delete
B) Create, Run, Update, Drop
C) Copy, Read, Update, Delete
D) Create, Retrieve, Use, Drop
✅ **Answer: A) Create, Read, Update, Delete**

80. Which MongoDB operation represents the "R" in CRUD?
A) `insert()`
B) `find()`
C) `update()`
D) `remove()`
✅ **Answer: B) `find()`**

81–100

81. Which MongoDB feature is used for processing data through multiple stages?
A) Aggregation pipeline
B) Data pipeline
C) Query chain
D) Processing queue
✅ **Answer: A) Aggregation pipeline**

82. Which aggregation stage filters documents?
A) `$filter`
B) `$match`
C) `$where`
D) `$select`
✅ **Answer: B) `$match`**

83. Which aggregation stage groups documents?
A) `$group`
B) `$collect`
C) `$cluster`
D) `$combine`
✅ **Answer: A) `$group`**

84. Which aggregation stage sorts documents?
A) `$order`

B) `$sort`
C) `$arrange`
D) `$sequence`
✅ **Answer: B) `$sort`**

85. Which aggregation stage limits the number of documents?
A) `$limit`
B) `$restrict`
C) `$max`
D) `$top`
✅ **Answer: A) `$limit`**

86. Which aggregation stage skips documents?
A) `$skip`
B) `$ignore`
C) `$offset`
D) `$jump`
✅ **Answer: A) `$skip`**

87. Which aggregation stage is used to calculate totals or averages within groups?
A) `$group`
B) `$calculate`
C) `$aggregate`
D) `$compute`
✅ **Answer: A) `$group`**

88. Which accumulator calculates the sum?
A) `$total`
B) `$sum`
C) `$addAll`
D) `$plus`
✅ **Answer: B) `$sum`**

89. Which accumulator calculates an average?
A) `$average`
B) `$avg`
C) `$mean`
D) `$averageValue`
✅ **Answer: B) `$avg`**

90. Which accumulator finds the maximum value?
A) `$maximum`
B) `$max`
C) `$highest`
D) `$topValue`
✅ **Answer: B) `$max`**

91. Which accumulator finds the minimum value?

A) `$minimum`

B) `$min`

C) `$lowest`

D) `$bottomValue`

✅ **Answer: B) `$min`**

92. Which accumulator counts values/documents in an aggregation group?

A) `$count`

B) `$number`

C) `$totalCount`

D) `$countAll`

✅ **Answer: A) `$count`**

93. Which aggregation stage can reshape documents and select fields?

A) `$project`

B) `$reshape`

C) `$select`

D) `$fields`

✅ **Answer: A) `$project`**

94. Which aggregation stage can rename or add computed fields?

A) `$set`

B) `$renameField`

C) `$change`

D) `$modify`

✅ **Answer: A) `$set`**

95. Which aggregation stage is used to join documents from another collection?

A) `$join`

B) `$lookup`

C) `$connect`

D) `$mergeJoin`

✅ **Answer: B) `$lookup`**

96. Which aggregation stage deconstructs an array field?

A) `$explode`

B) `$unwind`

C) `$flatten`

D) `$expand`

✅ **Answer: B) `$unwind`**

97. What does `$unwind` do?

A) Deletes an array

B) Creates a separate document for each array element

C) Sorts an array

D) Combines arrays
✅ **Answer: B) Creates a separate document for each array element**

98. MongoDB supports relationships between documents primarily through:
A) Joins only
B) Embedding and referencing
C) Foreign keys only
D) Stored procedures only
✅ **Answer: B) Embedding and referencing**

99. Embedding means:
A) Storing related data inside the same document
B) Storing data in another database
C) Encrypting documents
D) Creating an index
✅ **Answer: A) Storing related data inside the same document**

100. Referencing means:
A) Storing related data using references to other documents
B) Copying every document
C) Deleting related documents
D) Creating duplicate collections
✅ **Answer: A) Storing related data using references to other documents**

101–120

101. Which approach is often preferred when related data is frequently accessed together?
A) Embedding
B) Always referencing
C) Separate databases
D) No relationship
✅ **Answer: A) Embedding**

102. Which approach can be useful when related data is large or shared by many documents?
A) Embedding only
B) Referencing
C) Duplication only
D) Array flattening
✅ **Answer: B) Referencing**

103. What is denormalization in MongoDB?
A) Combining related data into documents to optimize reads
B) Removing all data
C) Converting MongoDB to SQL
D) Creating only normalized tables
✅ **Answer: A) Combining related data into documents to optimize reads**

104. MongoDB supports transactions across multiple documents in:
A) Modern MongoDB versions
B) No MongoDB version
C) Only relational databases
D) Only Redis
✅ **Answer: A) Modern MongoDB versions**

105. A MongoDB transaction provides:
A) Atomicity across the operations in the transaction
B) Automatic deletion
C) Automatic indexing of every field
D) Schema enforcement for all collections
✅ **Answer: A) Atomicity across the operations in the transaction**

106. What is replication in MongoDB?
A) Maintaining copies of data across multiple servers
B) Compressing data
C) Removing duplicate documents
D) Creating database schemas
✅ **Answer: A) Maintaining copies of data across multiple servers**

107. MongoDB replication is implemented using a:
A) Replica set
B) Cluster table
C) Data group
D) Copy set
✅ **Answer: A) Replica set**

108. In a typical MongoDB replica set, the primary node:
A) Receives writes
B) Only performs backups
C) Cannot read data
D) Stores no data
✅ **Answer: A) Receives writes**

109. Secondary nodes in a replica set:
A) Replicate data from the primary
B) Cannot store data
C) Replace all databases
D) Delete primary data
✅ **Answer: A) Replicate data from the primary**

110. If the primary MongoDB node fails, the replica set can:
A) Elect a new primary
B) Delete all data
C) Stop permanently

D) Remove all secondary nodes
✅ **Answer: A) Elect a new primary**

111. What is sharding in MongoDB?
A) Horizontal partitioning of data across servers
B) Vertical compression of documents
C) Encrypting a database
D) Removing indexes
✅ **Answer: A) Horizontal partitioning of data across servers**

112. The key used to distribute documents across shards is called:
A) Partition key
B) Shard key
C) Distribution key
D) Cluster key
✅ **Answer: B) Shard key**

113. Sharding is primarily used to:
A) Scale large datasets horizontally
B) Remove all replication
C) Replace BSON
D) Enforce relational schemas
✅ **Answer: A) Scale large datasets horizontally**

114. Which of the following is a MongoDB GUI tool?
A) MongoDB Compass
B) MySQL Workbench
C) pgAdmin
D) SQL Developer
✅ **Answer: A) MongoDB Compass**

115. MongoDB Compass is used for:
A) Graphical interaction with MongoDB databases
B) Compiling Java programs
C) Managing Linux processes
D) Designing HTML pages
✅ **Answer: A) Graphical interaction with MongoDB databases**

116. Which programming language is MongoDB originally implemented primarily in?
A) C++
B) Java
C) Python
D) PHP
✅ **Answer: A) C++**

117. Which driver can be used to connect Python applications to MongoDB?
A) PyMongo

B) PySQL
C) MongoPySQL
D) PythonDB
✅ **Answer: A) PyMongo**

118. Which Python class is commonly used to create a MongoDB connection?
A) `MongoClient`
B) `MongoConnection`
C) `DatabaseClient`
D) `MongoDBClient`
✅ **Answer: A) `MongoClient`**

119. In PyMongo, which method is commonly used to insert one document?
A) `insert_one()`
B) `insertOne()`
C) `add_one()`
D) `create_one()`
✅ **Answer: A) `insert_one()`**

120. In PyMongo, which method is commonly used to insert multiple documents?
A) `insert_many()`
B) `insertMany()`
C) `add_many()`
D) `create_many()`
✅ **Answer: A) `insert_many()`**

121–144

121. In PyMongo, which method retrieves one document?
A) `find_one()`
B) `findOne()`
C) `get_one()`
D) `read_one()`
✅ **Answer: A) `find_one()`**

122. In PyMongo, `find()` generally returns:
A) A cursor
B) A string
C) A single document only
D) A database name
✅ **Answer: A) A cursor**

123. Which Python package provides the `ObjectId` class?
A) `bson`
B) `json`
C) `mongo`

D) `pymongo.json`
✅ **Answer: A) `bson`**

124. What is an ObjectId in MongoDB?
A) A unique identifier commonly used for `_id`
B) A database name
C) A collection name
D) An encryption key
✅ **Answer: A) A unique identifier commonly used for `_id`**

125. How many bytes is a standard MongoDB ObjectId?
A) 8 bytes
B) 12 bytes
C) 16 bytes
D) 24 bytes
✅ **Answer: B) 12 bytes**

126. When represented as a hexadecimal string, an ObjectId is commonly:
A) 12 characters
B) 16 characters
C) 24 characters
D) 32 characters
✅ **Answer: C) 24 characters**

127. Which MongoDB shell command can be used to insert a document?
A) `db.collection.insertOne({...})`
B) `db.collection.add({...})`
C) `db.collection.create({...})`
D) `db.collection.put({...})`
✅ **Answer: A) `db.collection.insertOne({...})`**

128. Which MongoDB method is used to update a document using a filter and update document?
A) `updateOne()`
B) `changeOne()`
C) `modify()`
D) `edit()`
✅ **Answer: A) `updateOne()`**

129. Which operator is commonly used with `updateOne()` to change a field without replacing the whole document?
A) `$set`
B) `$change`
C) `$modify`
D) `$update`
✅ **Answer: A) `$set`**

130. What happens if an update operation uses `$set` on a field that does not exist?
A) The field is created
B) The operation always fails
C) The entire document is deleted
D) The collection is deleted
✅ **Answer: A) The field is created**

131. Which method replaces an entire document?
A) `replaceOne()`
B) `replaceDocument()`
C) `overwriteOne()`
D) `substituteOne()`
✅ **Answer: A) `replaceOne()`**

132. Which MongoDB operation removes one document matching a filter?
A) `deleteOne()`
B) `removeDocument()`
C) `eraseOne()`
D) `dropOne()`
✅ **Answer: A) `deleteOne()`**

133. Which MongoDB operation removes all documents matching a filter?
A) `deleteMany()`
B) `deleteAll()`
C) `removeAll()`
D) `dropMany()`
✅ **Answer: A) `deleteMany()`**

134. Which command displays the databases available to the current MongoDB user?
A) `show dbs`
B) `show databases`
C) `list dbs`
D) `db.list()`
✅ **Answer: A) `show dbs`**

135. MongoDB is commonly described as:
A) A document-oriented NoSQL database
B) A pure relational database
C) A spreadsheet application
D) An operating system
✅ **Answer: A) A document-oriented NoSQL database**

136. Which format is closest to the syntax used to represent MongoDB documents?
A) JSON
B) CSV
C) HTML

D) SQL
✅ **Answer: A) JSON**

137. Which statement about MongoDB schema is generally correct?
A) Every document in a collection must have exactly the same fields
B) Documents in a collection can have different fields
C) MongoDB requires foreign keys
D) MongoDB requires fixed-width rows
✅ **Answer: B) Documents in a collection can have different fields**

138. Which of the following is a valid MongoDB document?
A) `{ name: "Alan", age: 20 }`
B) `SELECT name FROM student`
C) `<student><name>Alan</name></student>`
D) `name = Alan; age = 20;`
✅ **Answer: A) `{ name: "Alan", age: 20 }`**

139. Which of the following is a valid MongoDB query to find students with age 20?
A) `db.students.find({age: 20})`
B) `db.students.select(age = 20)`
C) `SELECT * FROM students WHERE age = 20`
D) `db.students.get(age:20)`
✅ **Answer: A) `db.students.find({age: 20})`**

140. Which query finds documents where age is greater than 18?
A) `db.students.find({age: {$gt: 18}})`
B) `db.students.find({age > 18})`
C) `db.students.find(age > 18)`
D) `db.students.select({age: 18})`
✅ **Answer: A) `db.students.find({age: {$gt: 18}})`**

141. In Spring Data MongoDB, which annotation marks a class as a MongoDB document?
A) `@Document`
B) `@Table`
C) `@Entity-SQL`
D) `@Collection-Row`
✅ **Answer: A) `@Document`**

142. Why must a PyMongo ObjectId usually be converted to a string before returning documents as JSON in Flask?
A) JSON does not allow letters
B) ObjectId is not JSON serializable by default
C) MongoDB deletes ObjectIds on read
D) Flask cannot read the `_id` field
✅ **Answer: B) ObjectId is not JSON serializable by default**

143. In a web application, a MongoClient instance should typically be:
A) Created and closed for every single query
B) Created inside every HTML template
C) Created once and reused (connection pooling)
D) Avoided in favour of raw sockets
✅ **Answer: C) Created once and reused (connection pooling)**

144. Which of the following is the recommended approach for MongoClient in a web application?
A) Create a new client for every request
B) Create a new client for every query
C) Reuse a MongoClient instance because it manages connection pooling
D) Never use MongoClient
✅ **Answer: C) Reuse a MongoClient instance because it manages connection pooling**

🔥 Highest-priority topics to revise

For your exam, I'd especially hammer these:

Priority	Topic	Focus
🔴 **Very High**	MongoDB basics & architecture	**95%**
🔴 **Very High**	CRUD operations	**95%**
🔴 **Very High**	MongoDB operators (`$gt`, `$lt`, `$in`, `$and`, etc.)	**95%**
🔴 **Very High**	Aggregation pipeline	**90%**
🔴 **Very High**	Indexing	**90%**
🟠 **High**	Replication & Replica Sets	**85%**
🟠 **High**	Sharding	**85%**
🟠 **High**	Embedding vs Referencing	**85%**
🟡 **Medium**	PyMongo	**75%**
🟡 **Medium**	ObjectId / BSON	**75%**
🟡 **Medium**	CAP theorem	**70%**
🟢 **Lower**	Spring Data MongoDB / Flask integration	**50–60%**

Exam hack: If you're short on time, don't try to memorize all 144 blindly. First learn the **MongoDB command patterns** (`find`, `insertOne`, `updateOne`, `$set`, `$gt`, `$in`, `$group`, `$match`, `$lookup`, etc.). Once those click, a *lot* of these MCQs become freebies. 😎

> Provide in theory reading formats where i should able to understand the concepts and other things

Absolutely bro 😎 — **this is a much better way to study** than memorizing 144 answers one by one.

Instead of giving you:

> Q1 → A
> Q2 → D
> Q3 → C

I'll turn the whole Unit V into **theory-style study notes**, so that when you read a concept, you understand **what it means, why it is used, important commands, differences, and the MCQ points that can be asked from it**.

📘 UNIT V — NoSQL & MongoDB

Complete Theory Reading Notes

---

1. What is NoSQL?

NoSQL stands for **"Not Only SQL."**

NoSQL databases are databases designed to store and process large amounts of data without depending completely on the traditional relational database model.

Traditional relational databases such as MySQL store data in:

> **Database → Tables → Rows → Columns**

NoSQL databases use different models depending on the type of data.

Why was NoSQL introduced?

Traditional relational databases can become difficult to scale when applications handle:

- Huge amounts of data
- Millions of users
- Rapidly changing data
- Unstructured or semi-structured data
- Very high numbers of requests

NoSQL databases were designed to provide:

- **Flexible schema**
- **Horizontal scalability**
- High availability

- Fast data access
- Handling of large distributed datasets

Simple example

A relational database might store a student as:

ID	Name	Age	Department
101	Ravi	20	CSE

MongoDB could store:

```
{
    "_id": 101,
    "name": "Ravi",
    "age": 20,
    "department": "CSE"
}
```

The MongoDB document can also contain additional fields without changing a predefined table structure.

---

2. Types of NoSQL Databases

There are four major types of NoSQL databases.

1. Key-Value Database

Data is stored as:

> **Key → Value**

Example:

```
101 → "Ravi"
102 → "Alan"
103 → "John"
```

The key uniquely identifies the value.

Example: Redis

2. Document Database

Data is stored as documents, generally using JSON-like structures.

Example:

```
{
    "name": "Alan",
    "age": 20,
    "course": "CSE"
}
```

Example: MongoDB

3. Wide-Column Database

Data is organized using column families rather than traditional relational tables.

Example: Apache Cassandra

4. Graph Database

Data is represented using:

- Nodes
- Relationships/Edges
- Properties

Example:

```
Alan → FRIEND_OF → John
```

Example: Neo4j

⭐ Remember

Type	Example
Key-Value	Redis
Document	MongoDB
Wide-Column	Cassandra
Graph	Neo4j

MCQ trap: MongoDB is a **document-oriented database**, not a relational database.

---

3. NoSQL vs Relational Database

Relational Database

Data is stored in tables.

Example:

```
Student Table

---

ID | Name | Age

---

1  | Alan | 20
2  | Ravi | 21
```

Tables usually follow a predefined schema.

NoSQL Database

Data can be stored as flexible documents.

```
{
    "name": "Alan",
    "age": 20
}
```

Another document can contain a different field:

```
{
    "name": "Ravi",
    "age": 21,
    "department": "CSE"
}
```

That’s one of the major advantages of NoSQL.

Important comparison

Relational DB	NoSQL
Tables	Documents / Key-value / Graph / Columns
Fixed schema commonly used	Flexible schema
SQL	Database-specific APIs/query languages
Vertical scaling traditionally common	Horizontal scaling commonly used
JOINs are common	Embedding/referencing often used

---

4. CAP Theorem

The **CAP theorem** is extremely important for NoSQL MCQs. 🔥

CAP stands for:

- **C — Consistency**
- **A — Availability**
- **P — Partition Tolerance**

The theorem states that a distributed system cannot simultaneously guarantee all three properties under a network partition.

C — Consistency

Consistency means that every read receives the most recent write or an error.

Example:

If you update:

```
Balance = ₹5000
```

then another user reading the data should not receive an older value such as:

```
Balance = ₹3000
```

when strong consistency is required.

A — Availability

Availability means every request receives a response.

The system continues responding even when some parts of the system have problems.

P — Partition Tolerance

Partition tolerance means the system continues operating even when communication between distributed nodes is interrupted.

Imagine:

```
Server A  X  Server B
```

The network connection between them fails.

A partition-tolerant system is designed to continue operating despite this failure.

⭐ Easy memory trick

CAP =

> **C**onsistency
> **A**vailability
> **P**artition tolerance

The CAP theorem was proposed by **Eric Brewer**.

---

5. What is MongoDB?

MongoDB is a **NoSQL, document-oriented database management system**.

Instead of storing information in rows and columns, MongoDB stores information as **documents**.

MongoDB documents are stored internally as **BSON**.

MongoDB hierarchy

Think of it like this:

```
MongoDB Server
      ↓
   Database
      ↓
 Collection
      ↓
 Document
      ↓
 Fields
```

This is one of the most important structures to remember.

---

6. MongoDB Database

A MongoDB server can contain multiple databases.

For example:

```
CollegeDB
BankDB
HospitalDB
LibraryDB
```

A database contains collections.

```
CollegeDB
   ├── students
   ├── faculty
   └── courses
```

---

7. MongoDB Collection

A **collection** is a group of MongoDB documents.

It is roughly comparable to a **table** in a relational database.

Example:

```
students
```

may contain:

```
Document 1
{
    "name": "Alan",
    "age": 20
}

Document 2
{
    "name": "Ravi",
    "age": 21
}
```

Unlike a traditional relational table, documents within a collection can have different fields.

---

8. MongoDB Document

A document is a record stored inside a collection.

Example:

```
{
    "_id": 101,
    "name": "Alan",
    "age": 20,
    "department": "CSE"
}
```

A document consists of **fields and values**.

For example:

```
name       → Alan
age        → 20
department → CSE
```

Documents resemble JSON.

---

9. BSON

MongoDB stores documents in **BSON**.

BSON means:

> **Binary JSON**

BSON extends JSON by supporting additional data types such as:

- Date
- Binary data
- ObjectId
- Decimal types

Important distinction

Don't confuse:

> MongoDB uses **JSON-like documents**

with:

> MongoDB internally stores documents as **BSON**

⭐ Exam point

MongoDB → BSON → Binary JSON

---

10. `_id` Field

Every MongoDB document normally has a unique `_id` field.

Example:

```
{
    "_id": 101,
    "name": "Alan"
}
```

If you don't provide `_id`, MongoDB automatically generates one.

Usually this is an **ObjectId**.

Example:

```
ObjectId("507f1f77bcf86cd799439011")
```

The `_id` field acts as the unique identifier of a document.

---

11. ObjectId

ObjectId is commonly used as MongoDB's default identifier.

A standard ObjectId is:

> **12 bytes**

When represented as hexadecimal text, it is:

> **24 hexadecimal characters**

Example:

```
507f1f77bcf86cd799439011
```

Remember:

12 bytes → 24 hexadecimal characters

This is a classic MCQ. 🎯

---

12. MongoDB Shell Commands

Some basic MongoDB commands are very important.

Show databases

```
show dbs
```

Select a database

```
use college
```

If the database doesn't already exist, MongoDB switches to that database context and it can be created when data is stored.

Display current database

```
db
```

Show collections

```
show collections
```

---

13. CRUD Operations

CRUD is one of the **highest-priority topics**.

CRUD stands for:

> **C — Create**
> **R — Read**
> **U — Update**
> **D — Delete**

---

14. Create — Insert Documents

To insert one document:

```
db.students.insertOne({
    name: "Alan",
    age: 20
})
```

To insert multiple documents:

```
db.students.insertMany([
    {
        name: "Alan",
        age: 20
    },
    {
        name: "Ravi",
        age: 21
    }
])
```

Remember

```
insertOne()  → one document
insertMany() → multiple documents
```

---

15. Read — Finding Documents

The most important read operation is:

```
find()
```

Example:

```
db.students.find()
```

This retrieves documents from the collection.

To find a specific document:

```
db.students.find({
    age: 20
})
```

---

16. `findOne()`

If you want one matching document:

```
db.students.findOne({
    name: "Alan"
})
```

Difference

```
find()     → returns matching documents/cursor
findOne()  → returns one matching document
```

---

17. Update Operations

MongoDB provides:

```
updateOne()
updateMany()
```

Example:

```
db.students.updateOne(
    {name: "Alan"},
    {$set: {age: 21}}
)
```

This searches for Alan and changes his age.

For multiple documents:

```
db.students.updateMany(
    {department: "CSE"},
    {$set: {college: "ABC College"}}
)
```

---

18. `$set` Operator

`$set` is used to assign or modify a field.

Example:

```
{
    $set: {
        age: 21
    }
}
```

If the field doesn't already exist, `$set` can create it.

---

19. `$inc` Operator

`$inc` increases or decreases a numeric value.

Example:

```
{$inc: {age: 1}}
```

If:

```
age = 20
```

after the operation:

```
age = 21
```

It can also decrease:

```
{$inc: {age: -1}}
```

---

20. `$mul` Operator

`$mul` multiplies a numeric field.

Example:

```
{$mul: {salary: 2}}
```

If salary is:

```
25000
```

it becomes:

```
50000
```

---

21. `$unset`

`$unset` removes a field from a document.

Example:

```
{$unset: {age: ""}}
```

The age field is removed.

Remember

```
$set   → create/change field
$unset → remove field
```

---

22. Delete Operations

To delete one document:

```
db.students.deleteOne({
    name: "Alan"
})
```

To delete multiple matching documents:

```
db.students.deleteMany({
    age: 20
})
```

Important

```
deleteOne()  → one matching document
deleteMany() → all matching documents
```

---

23. Dropping a Collection

To remove an entire collection:

```
db.students.drop()
```

This is different from `deleteMany()`.

`deleteMany({})`

Deletes documents.

`drop()`

Deletes the collection itself.

Think:

> **deleteMany → empty the room**
> **drop → demolish the room** 😂

---

24. Dropping a Database

To remove the current database:

```
db.dropDatabase()
```

This removes the database and its collections.

---

25. MongoDB Query Operators

MongoDB provides operators to perform conditions.

These are extremely important for MCQs.

`$eq`

Means:

> Equal to

```
{age: {$eq: 20}}
```

$ne

Means:

> Not equal to

```
{age: {$ne: 20}}
```

$gt

Means:

> Greater than

```
{age: {$gt: 18}}
```

Meaning:

```
age > 18
```

$gte

Means:

> Greater than or equal to

```
age >= 18
```

$lt

Means:

> Less than

```
age < 18
```

$lte

Means:

> Less than or equal to

```
age <= 18
```

🔥 Memorize this group

```
$gt  → >
$gte → >=
$lt  → <
$lte → <=
$eq  → =
$ne  → !=
```

---

26. `$in`

`$in` checks whether a value matches **any value from a specified list**.

Example:

```
{age: {$in: [18, 20, 22]}}
```

This matches age:

```
18 OR 20 OR 22
```

---

27. `$nin`

`$nin` means:

> Not in

Example:

```
{age: {$nin: [18, 20]}}
```

It matches documents where age is not 18 or 20.

---

28. Logical Operators

MongoDB supports logical operators such as:

- `$and`
- `$or`
- `$not`
- `$nor`

$and

Both conditions must be true.

Conceptually:

```
Condition A AND Condition B
```

Example:

```
{
    $and: [
        {age: {$gt: 18}},
        {department: "CSE"}
    ]
}
```

---

29. $or

At least one condition must be true.

```
Condition A OR Condition B
```

Example:

```
{
    $or: [
        {age: 18},
        {age: 20}
    ]
}
```

---

30. $not

Performs logical negation.

In simple terms:

> If a condition is true, `$not` makes it false, and vice versa.

---

31. $nor

`$nor` means:

> None of the specified conditions should be true.

It is basically the logical NOR operation.

---

32. `$exists`

Checks whether a field exists.

Example:

```
{
    phone: {$exists: true}
}
```

This finds documents that contain the `phone` field.

To find documents where the field doesn't exist:

```
{
    phone: {$exists: false}
}
```

---

33. `$regex`

`$regex` is used for pattern matching.

For example:

```
{
    name: {$regex: "^A"}
}
```

This can find names beginning with `A`.

Useful for text/pattern-based searches.

---

34. `$type`

`$type` checks the BSON data type of a field.

For example, you can query documents where a particular field is stored as a string or number.

---

35. Array Operators

MongoDB documents can contain arrays.

Example:

```
{
    name: "Alan",
    skills: ["C", "Java", "MongoDB"]
}
```

MongoDB provides operators for working with arrays.

Important ones include:

- `$all`
- `$size`
- `$elemMatch`
- `$push`
- `$addToSet`
- `$pop`
- `$pull`

---

36. `$all`

Checks whether an array contains **all specified values**.

Example:

```
{
    skills: {
        $all: ["Java", "MongoDB"]
    }
}
```

The document must contain both values.

---

37. `$size`

Checks the number of elements in an array.

Example:

```
{
    skills: {$size: 3}
}
```

This finds documents whose `skills` array contains exactly 3 elements.

---

38. `$elemMatch`

`$elemMatch` is useful when working with arrays containing objects.

It checks whether at least one array element satisfies specified conditions.

---

39. `$push`

Adds an element to an array.

Suppose:

```
skills: ["C", "Java"]
```

Using:

```
{$push: {skills: "MongoDB"}}
```

results in:

```
["C", "Java", "MongoDB"]
```

---

40. `$addToSet`

Also adds a value to an array, but it prevents duplicate values.

This is the important difference:

```
$push      → can add duplicates
$addToSet  → avoids duplicates
```

---

41. `$pop`

Removes an element from an array.

It can remove from:

- Beginning
- End

---

42. `$pull`

Removes matching elements from an array.

For example, if:

```
skills = ["C", "Java", "Python"]
```

and you pull `"Java"`:

```
skills = ["C", "Python"]
```

---

43. Sorting

MongoDB uses:

```
sort()
```

Example:

```
db.students.find().sort({age: 1})
```

1 means:

> Ascending

`-1` means:

> Descending

Remember

```
1  → Ascending ↑
-1 → Descending ↓
```

---

44. Limiting Results

The `limit()` method controls how many documents are returned.

Example:

```
db.students.find().limit(5)
```

This returns at most 5 documents.

---

45. Skipping Documents

The `skip()` method skips a specified number of documents.

Example:

```
db.students.find().skip(5)
```

This skips the first five documents in the result set.

`skip()` and `limit()` are commonly used for **pagination**.

---

46. Counting Documents

MongoDB provides:

```
countDocuments()
```

Example:

```
db.students.countDocuments({})
```

This returns the number of matching documents.

---

47. Projection

Projection means selecting only the fields you need from a document.

Suppose a document contains:

```
name
age
department
address
phone
email
```

But you only want:

```
name
age
```

You can use projection.

Example:

```
db.students.find(
    {},
    {name: 1, age: 1}
)
```

Projection rule

```
1 → include
0 → exclude
```

The `_id` field is a special case and is commonly explicitly excluded when needed:

```
{_id: 0, name: 1}
```

---

48. Indexing

An **index** is a data structure that helps MongoDB find documents more efficiently.

Imagine a textbook with 500 pages.

Without an index:

> You might search page by page.

With an index:

> You can jump directly to the relevant page.

MongoDB indexes work on a similar principle.

Main purpose

> **Improve query performance.**

---

49. Creating an Index

MongoDB uses:

```
createIndex()
```

Example:

```
db.students.createIndex({name: 1})
```

This creates an ascending index on `name`.

---

50. Unique Index

A unique index prevents duplicate indexed values.

For example, if email must be unique:

```
db.students.createIndex(
    {email: 1},
    {unique: true}
)
```

Then two students cannot have the same indexed email.

---

51. Text Index

A **text index** is used for text-search operations.

It is useful when applications need to search text content.

---

52. Hashed Index

A hashed index generates hash values for indexed data.

Hashed indexes are especially associated with certain **sharding** strategies.

---

53. TTL Index

TTL stands for:

> **Time To Live**

A TTL index allows MongoDB to automatically remove documents after a specified amount of time.

This is useful for temporary data such as:

- Sessions
- Temporary records
- Expiring information

---

54. Aggregation

Aggregation is one of the **most important MongoDB topics**. 🔥

Aggregation is used when you want to:

- Calculate totals
- Calculate averages
- Group data
- Filter data
- Sort data
- Transform documents
- Perform more complex analysis

MongoDB performs aggregation using an:

> **Aggregation Pipeline**

---

55. Aggregation Pipeline

Think of an aggregation pipeline as a sequence of processing stages.

```
Documents
    ↓
$match
    ↓
$group
    ↓
$sort
    ↓
$project
    ↓
Result
```

Each stage receives the output of the previous stage.

---

56. `$match`

`$match` filters documents.

It is conceptually similar to:

```
WHERE
```

Example:

```
{$match: {department: "CSE"}}
```

Only CSE documents continue to the next stage.

---

57. `$group`

`$group` groups documents based on a specified field.

It can be used to calculate:

- Sum
- Average
- Minimum
- Maximum
- Count

Example concept:

```
CSE → 100 students
ECE → 80 students
EEE → 70 students
```

---

58. `$sum`

`$sum` calculates a total.

For example, it can calculate total salary:

```
₹20,000
+ ₹30,000
+ ₹40,000

---

₹90,000
```

---

59. `$avg`

`$avg` calculates the average.

Example:

```
10 + 20 + 30 = 60
60 / 3 = 20
```

So:

```
Average = 20
```

---

60. `$max`

`$max` returns the largest value.

Example:

```
10, 50, 20
```

Maximum:

```
50
```

---

61. `$min`

`$min` returns the smallest value.

Example:

```
10, 50, 20
```

Minimum:

```
10
```

---

62. `$count`

`$count` counts documents/values at an aggregation stage.

It is useful when determining how many documents belong to a group or satisfy a pipeline.

---

63. `$project`

`$project` controls the fields that appear in the aggregation result.

It can:

- Include fields
- Exclude fields
- Rename fields
- Create calculated fields

---

64. `$set`

In aggregation, `$set` can add or modify fields while processing documents.

It is useful for creating calculated or transformed fields.

---

65. `$sort`

`$sort` sorts aggregation results.

Again:

```
1  → Ascending
-1 → Descending
```

---

66. `$limit`

Limits the number of documents passing through the pipeline.

Example concept:

```
100 documents
      ↓
$limit: 10
      ↓
10 documents
```

---

67. `$skip`

Skips a specified number of documents in the aggregation pipeline.

Useful for pagination and controlled result processing.

---

68. `$unwind`

`$unwind` is used with arrays.

Suppose:

```
{
    name: "Alan",
    skills: ["Java", "C", "MongoDB"]
}
```

After `$unwind`, MongoDB can create separate pipeline documents for each array element.

Conceptually:

```
Alan → Java
Alan → C
Alan → MongoDB
```

⭐ Remember

> **`$unwind` → breaks an array into separate documents.**

---

69. `$lookup`

`$lookup` is used to combine information from another collection.

It is similar to a JOIN concept in relational databases.

For example:

```
students collection
        ↓
     $lookup
        ↓
courses collection
```

This allows related information to be combined.

Important

> `$lookup` → MongoDB's aggregation-stage mechanism for joining related data.

---

70. Embedding

MongoDB allows related data to be stored inside the same document.

Example:

```
{
    name: "Alan",
    address: {
        city: "Chennai",
        state: "Tamil Nadu"
    }
}
```

The address is embedded inside the student document.

Advantage

If the address is usually accessed with the student, embedding can be convenient and efficient.

---

71. Referencing

Instead of embedding related data, we can store a reference to another document.

Conceptually:

```
Student
   ↓
course_id
   ↓
Course Document
```

This is useful when the related data:

- Is large
- Changes independently
- Is shared by many documents

---

72. Embedding vs Referencing

Embedding	Referencing
Related data stored together	Related data stored separately
Faster access for related data in many cases	Useful for shared/large data
Can make documents larger	Requires additional lookup/application logic
Good for tightly related data	Good for loosely related data

Easy rule

> **Frequently used together → Embedding**

> **Independent/shared/large → Referencing**

---

73. Denormalization

Denormalization means storing related information together or duplicating some information to make reads faster.

Traditional relational databases often emphasize normalization.

MongoDB frequently allows denormalized document structures when they suit application access patterns.

Main goal

> **Optimize data access and reduce expensive joins/lookups.**

---

74. Transactions

A transaction is a group of operations treated as one logical unit.

Suppose a bank transfer involves:

```
Account A → subtract ₹1000
Account B → add ₹1000
```

Both operations should succeed together.

If one fails, the transaction can roll back.

MongoDB supports transactions in modern versions, including multi-document transactions.

---

75. Replication

Replication means maintaining multiple copies of data.

MongoDB uses:

> **Replica Sets**

Example:

```
            Primary
           /       \
          ↓         ↓
    Secondary   Secondary
```

The primary generally handles writes, while secondaries replicate data from the primary.

---

76. Primary Node

The **primary** is the main member that receives writes in a typical replica set.

Example:

```
Application
     ↓
  Primary
```

---

77. Secondary Node

Secondary nodes replicate data from the primary.

```
Primary
   ↓
Secondary
   ↓
Data copy
```

They provide redundancy and can participate in read operations depending on configuration.

---

78. Automatic Failover

One of the important benefits of replica sets is failover.

Suppose:

```
Primary 💥
```

The replica set can elect another eligible member as the new primary.

Conceptually:

```
Old Primary
     ↓
   FAILED
     ↓
Election
     ↓
New Primary
```

This helps maintain availability.

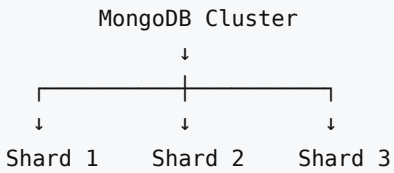
---

79. Sharding

Sharding is used for **horizontal scaling**.

Instead of storing the entire dataset on one server, data can be distributed across multiple servers.

Example:



Each shard stores part of the overall dataset.

---

80. Shard Key

MongoDB uses a **shard key** to determine how documents are distributed across shards.

The choice of shard key is extremely important because a poor shard key can lead to:

- Uneven distribution
- Performance problems
- Hotspots

Remember

> **Shard key → determines data distribution across shards.**

---

81. Replication vs Sharding

This is a very common conceptual question.

Replication

Main purpose:

> **Availability + redundancy**

Multiple copies of the data exist.

Sharding

Main purpose:

> **Horizontal scalability**

Data is distributed across multiple servers.

Easy memory trick

```
Replication → Copies
Sharding    → Splits
```

🔥 **Copies vs Splits** — remember this and you'll never confuse them.

---

82. MongoDB Compass

MongoDB Compass is a graphical user interface for MongoDB.

Instead of using only the command line, you can use Compass to:

- View databases
- View collections
- View documents
- Run queries
- Create indexes
- Analyze data

Think:

> **MongoDB Compass = GUI for MongoDB**

---

83. MongoDB with Python

Python applications can communicate with MongoDB using:

> **PyMongo**

PyMongo is the commonly used Python driver for MongoDB.

---

84. MongoClient

In PyMongo, `MongoClient` is commonly used to connect to MongoDB.

Conceptually:

```
Python Application
        ↓
   MongoClient
        ↓
     MongoDB
```

Example:

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
```

---

85. PyMongo CRUD

Python equivalents are commonly written as:

```
insert_one()
insert_many()

find_one()
find()

update_one()
update_many()

delete_one()
delete_many()
```

Notice the naming difference from the MongoDB shell.

Shell

```
insertOne()
```

Python/PyMongo

```
insert_one()
```

The concept is the same.

---

86. Cursor in PyMongo

When using:

```
collection.find()
```

PyMongo generally returns a **cursor**.

A cursor allows the application to iterate through the matching documents.

Conceptually:

```
find()
 ↓
Cursor
 ↓
Document 1
Document 2
```

```
Document 3
...
```

---

87. ObjectId in Python

ObjectId is available through BSON-related functionality.

Conceptually:

```
from bson import ObjectId
```

ObjectId is commonly used when working with MongoDB document IDs.

---

88. MongoDB and JSON

MongoDB documents look similar to JSON.

Example:

```
{
    "name": "Alan",
    "age": 20
}
```

However:

> MongoDB internally stores documents as **BSON**, not plain JSON.

Important MCQ distinction

```
JSON → human-friendly data representation
BSON → binary representation used by MongoDB
```

---

89. MongoDB in Web Applications

MongoDB can be used as the database behind web applications.

Typical architecture:

```
Browser
   ↓
Frontend
   ↓
Backend / API
```

```
   ↓
MongoDB
```

For example:

```
HTML/CSS/JavaScript
        ↓
     Python
        ↓
      PyMongo
        ↓
     MongoDB
```

---

90. MongoClient Reuse

A MongoClient should generally be reused rather than creating a brand-new client for every query.

Why?

Because the MongoDB driver manages connections and connection pooling.

Bad approach

```
Query 1 → create client
Query 2 → create client
Query 3 → create client
```

Better approach

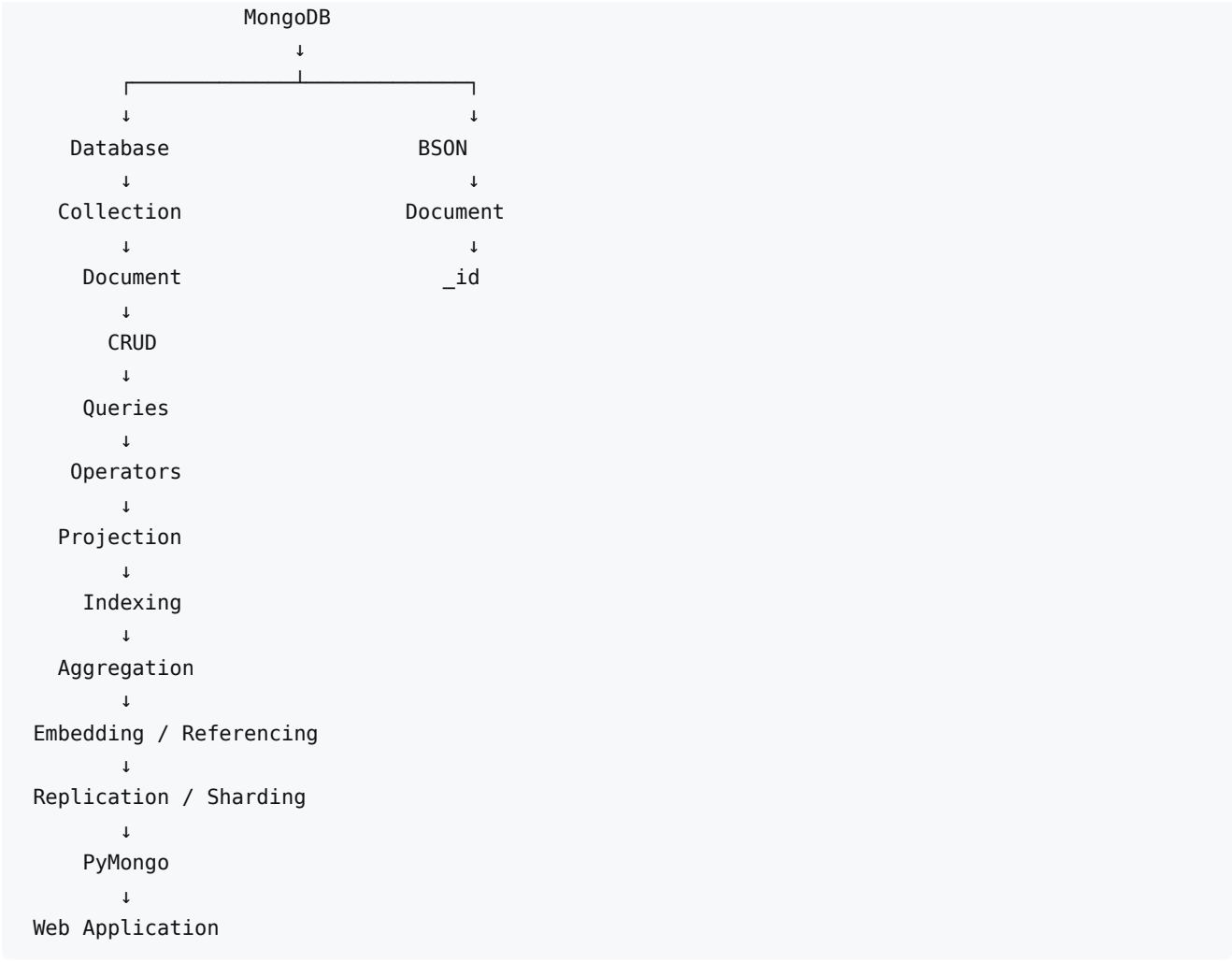
```
Application
     ↓
One reusable MongoClient
     ↓
Connection Pool
     ↓
MongoDB
```

This improves efficiency.

🧠 Final Revision Map

If you want to understand the **entire Unit V**, think about it as one connected story:

```
                  NoSQL
                    ↓
            Types of NoSQL
                    ↓
```



🔥 What you should know REALLY well

Before your exam, make sure you can explain these **without looking at notes**:

1. **What is NoSQL and why is it used?**
2. **Four types of NoSQL databases**
3. **MongoDB architecture**
4. **Database → Collection → Document → Field**
5. **JSON vs BSON**
6. **`_id` and ObjectId**
7. **CRUD operations**
8. **`find()` vs `findOne()`**
9. **`updateOne()` vs `updateMany()`**
10. **`deleteOne()` vs `deleteMany()`**
11. **`$gt`, `$gte`, `$lt`, `$lte`, `$eq`, `$ne`**
12. **`$in` and `$nin`**
13. **`$and`, `$or`, `$not`, `$nor`**
14. **Array operators**
15. **Projection**
16. **Indexes and their purpose**
17. **Aggregation pipeline**

18. **`$match`, `$group`, `$project`, `$sort`, `$unwind`, `$lookup`**
19. **Embedding vs Referencing**
20. **Replication and Replica Sets**
21. **Primary vs Secondary**
22. **Sharding and Shard Key**
23. **Replication vs Sharding**
24. **MongoDB Compass**
25. **PyMongo and MongoClient**

🎯 Best way to study this

Don’t try to memorize the definitions word-for-word. Read each section and ask yourself:

> **“If the examiner gives me a situation, do I know which MongoDB concept to use?”**

That’s the sweet spot for MCQs. Once you understand **why** something works, the four options start looking hilariously obvious. 😄